

DEV_IMPL_GUIDE - DD: BIL-02-READ-01 — Invoice Read API

1. Purpose

This guide explains how to implement DD_BIL-02-READ-01-Invoice_Read_API-v1.5.md as executable SOPHIX behavior.

Use it with:

- DD_BIL-02-READ-01-Invoice_Read_API-v1.5.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md when this DD participates in subscription workflow operations.
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md when this DD references worker topics.

2. Runtime Actors

Runtime component	Responsibility
API pod / module service	Receives requests, validates, loads authoritative state, starts commands.
Owning module database	Stores tables/configuration owned by this DD.
Reference modules	Provide data through approved APIs/client wrappers.
Camunda	Runs BPMN process when this DD is workflow-driven.
Worker apps	Execute logical worker topics.
Drools	Evaluates packages when rules are declared.
Kafka/outbox	Publishes success/failure and integration events.

3. Before Implementation

Confirm these contracts from the DD:

Contract	Value
Endpoint scan reference	GET /api/admin/invoices/download-stats?operator={code}
BPMN/process key	configured BPMN process
Drools package	configured Drools package
Tables	No CREATE TABLE block declared
Events	InvoiceCancelled, InvoiceIssued, InvoiceStatusChanged, PaymentApplied

4. Execution Flow

1. Apply migrations and seed configuration/catalog rows.
2. Implement the module service/repository layer around the tables and invariants.
3. Implement public/internal APIs from the DD.
4. Implement rule evaluation if the DD defines Drools packages.
5. Implement event production/consumption through outbox where the DD defines events.
6. Add integration tests for table constraints, API validation, and event payloads.

5. API And Command Handling

Endpoints found in the DD:

- GET /api/admin/invoices/download-stats?operator={code}
- GET /api/admin/invoices/pdf-render-failures?operator={code}

- GET /api/invoices
- GET /api/invoices/credit-notes
- GET /api/invoices/cycle
- GET /api/invoices/debit-notes
- GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW
- GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/pdf
- GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/status
- GET /api/invoices/pf_01HZX9M2K3LA8N7QR4S5T6U7VW
- GET /api/invoices/pro-forma
- GET /api/invoices/tax
- GET /api/invoices/{id}
- GET /api/invoices/{id}/pdf
- GET /api/invoices/{id}/status
- GET /api/invoices/{id}?lightweight=true
- GET /api/invoices?account_id=X&status=PENDING&sort=issue_date_desc&limit=20
- GET /api/invoices?customer_id=X&status=PENDING&sort=issue_date_desc&limit=20
- GET /api/invoices?operator_code=WIK&customer_id=cust_01HX...&issue_date_from=2026-01-01&issue_date_to=2026-05-31&limit=50
- GET /api/invoices?status=PENDING&sort=issue_date_desc&limit=20
- POST /api/admin/cache/invalidate
- POST /api/admin/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/rerender-pdf
- POST /api/admin/invoices/{id}/rerender-pdf

Implementation rules:

- Resolve actor user and role from auth context, not from public request body.
- Check idempotency before inserting a new operation or creating a side effect.
- Return the existing operation/result for the same idempotency key and same request hash.
- Return 409 IDEMPOTENCY_KEY_REUSED for the same key with a different request hash.
- Use transactions or unique constraints for concurrency-sensitive creation.

6. Data Loading

Load data in this order:

1. Authoritative aggregate state from the owning module.
2. Operator/module config from this DD's config tables.
3. Catalog/reference data from approved APIs or client wrappers.
4. Current in-flight operation state from the framework, where relevant.

Do not read current mutable state from cache for state-changing operations.

7. Drools Configuration

Packages found:

- No Drools package explicitly declared.

For each package:

- Define fact classes that mirror the request, current state, config, and reference data.

- Keep rule output as explicit decision fields.
- Unit-test accept and reject cases from the DD.
- BPMN should re-check critical rules after start if state may have changed.

8. BPMN And Worker Topics

Worker topics found:

- billing-invoicing
- billing-invoicing-read

Implementation rules:

- Model service tasks with `camunda:type="external"` and the exact topic string.
- Worker handler must validate required variables before calling downstream services.
- Store outputs as process variables with stable names.
- Use message correlation keys exactly as specified by the DD.
- Add compensation paths for each external side effect.

9. Events

Events found:

- InvoiceCancelled
- InvoiceIssued
- InvoiceStatusChanged
- PaymentApplied

Event rules:

- Emit success only after authoritative commit.
- Emit rejected/failure event only after compensation/final failure decision.
- Use the outbox table/dispatcher; do not publish directly inside the transaction boundary unless the platform standard allows it.

10. Smoke Tests

Add tests for:

- Happy path.
- Invalid role or trigger.
- Invalid state.
- Missing config/catalog row.
- Dependency failure.
- Retry/idempotency.
- Event payload correctness.
- Worker topic failure and compensation.

11. Common Mistakes

- Treating worker-topic names as shell commands.
- Querying another module's database directly.
- Reading current state from cache in a state-changing flow.

- Emitting success before final commit.
- Hiding manual recovery paths for partial external side effects.
- Creating one deployment per topic when a consolidated worker app would be easier to operate.